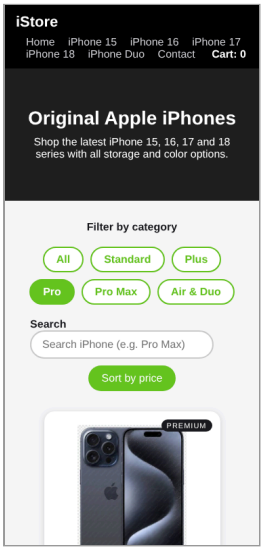
Web Technologies (CSC336) - Lab Mid Term

UMAR SOHAIL
Registration No: FA24-BCS-165

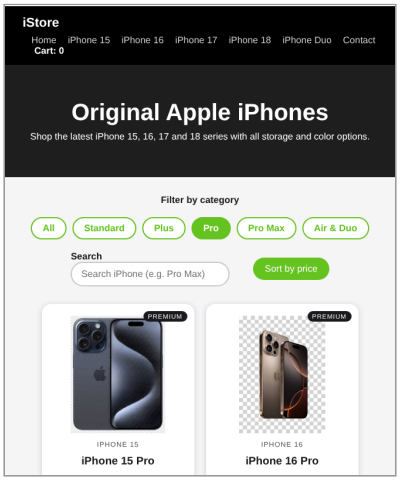
N = 65, BP = 665px, H = 95 (accent hsl(95, 70%, 45%)), K = 5, sticky offset = 95px, Premium limit = 6500

Task 1 - CSS-only category filter

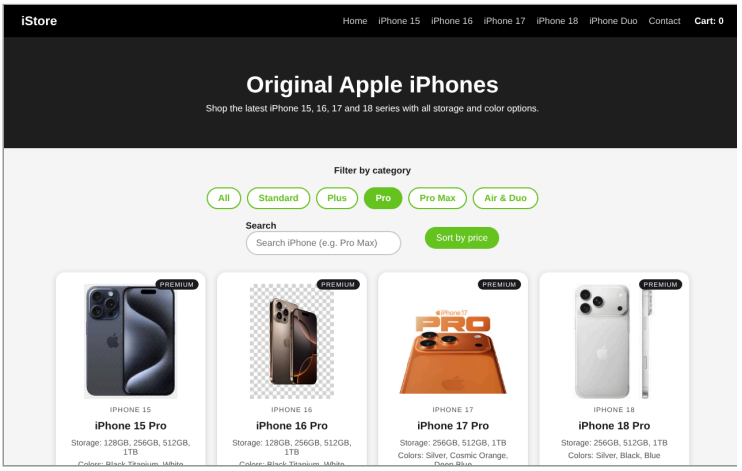
Screenshots with the "Pro" filter selected:



375px



665px



1280px

HTML added

```
<!-- TASK 1: K + 1 = 6 radios (All + 5 categories), "All" checked on load -->
    <form class="filter-form" aria-label="Filter by category">
      <fieldset class="filter-bar">
        <legend>Filter by category</legend>
        <label><input type="radio" name="category" id="f-all" value="all" checked> All</label>
        <label><input type="radio" name="category" id="f-standard" value="standard"> Standard</label>
        <label><input type="radio" name="category" id="f-plus" value="plus"> Plus</label>
        <label><input type="radio" name="category" id="f-pro" value="pro"> Pro</label>
        <label><input type="radio" name="category" id="f-promax" value="promax"> Pro Max</label>
        <label><input type="radio" name="category" id="f-special" value="special"> Air &amp;amp; Duo</label>
      </fieldset>
    </form>
```

CSS added

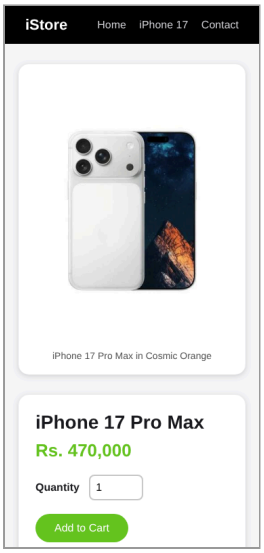
```
/* ===== TASK 1: CSS-only category filter ===== */
    .filter-bar {
      border: 0;
      display: flex;
      flex-wrap: wrap;
      justify-content: center;
      gap: 10px;
      padding: 30px 20px 10px;
    }
    .filter-bar legend {
      float: left;            /* keeps the legend on the same row as the buttons */
      width: 100%;
      text-align: center;
      font-weight: bold;
      margin-bottom: 10px;
    }
    .filter-bar label {
      position: relative;
      cursor: pointer;
      padding: 8px 18px;
```

```
      border: 2px solid var(--accent);
      border-radius: 20px;
      background: white;
      color: var(--accent);
      font-weight: bold;
    }
    /* the radio itself is hidden visually but stays keyboard-focusable */
    .filter-bar input {
      position: absolute;
      opacity: 0;
      width: 1px;
      height: 1px;
    }
    /* selected filter looks selected: accent colour H */
    .filter-bar label:has(input:checked) { background: var(--accent); color: white; }
    .filter-bar label:has(input:focus-visible) { outline: 3px solid #1d1d1f; outline-offset: 2px; }

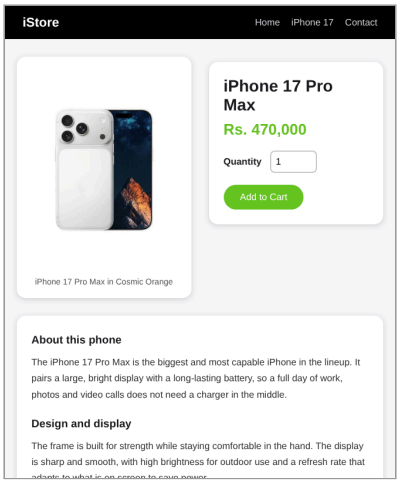
    /* hide every card that is NOT in the chosen category (display:none = no gaps in the flex grid) */
    .shop:has(#f-standard:checked) .card:not([data-category="standard"]),
    .shop:has(#f-plus:checked)     .card:not([data-category="plus"]),
    .shop:has(#f-pro:checked)      .card:not([data-category="pro"]),
    .shop:has(#f-promax:checked)   .card:not([data-category="promax"]),
    .shop:has(#f-special:checked)  .card:not([data-category="special"]) {
      display: none;
    }
```

Task 2 - Product page layout with a sticky details box

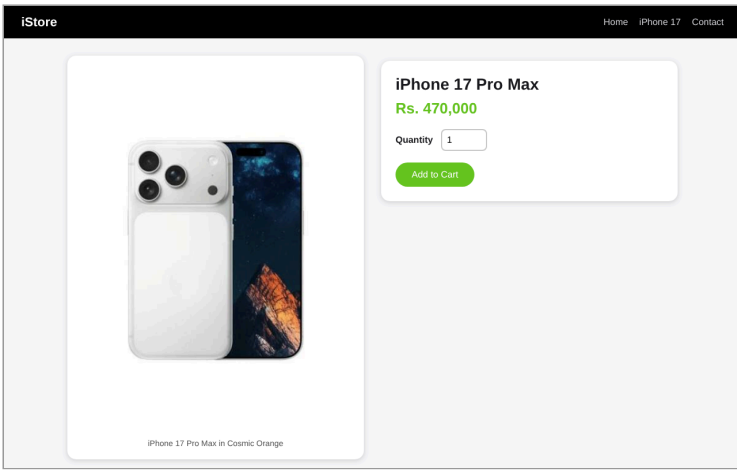
Top of the page:



375px

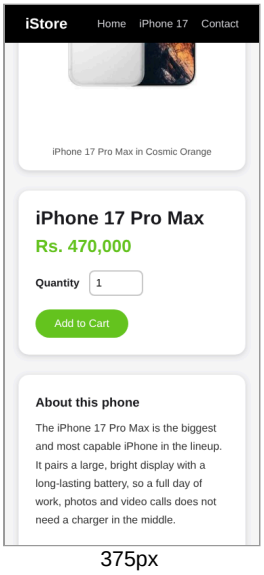


665px

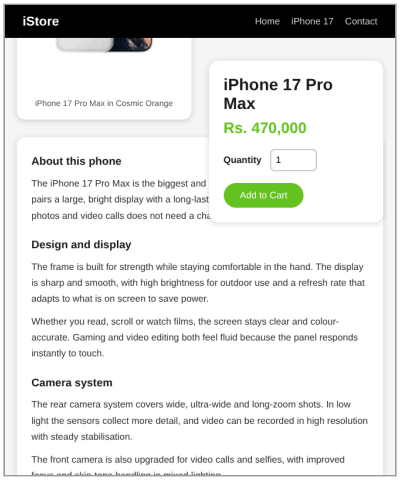


1280px

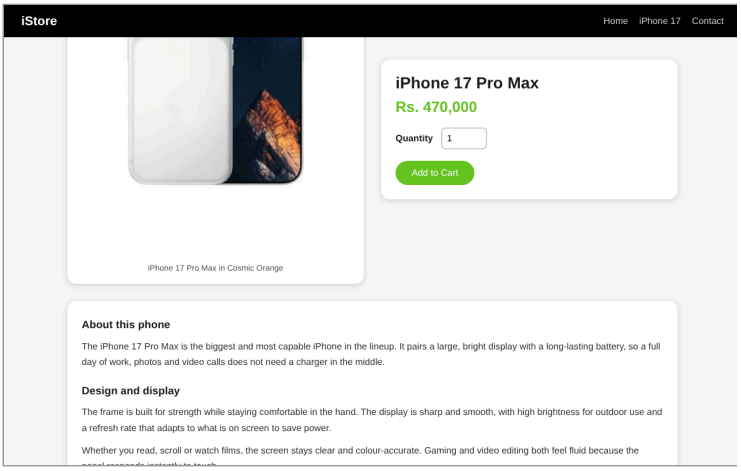
After scrolling (665px and 1280px: details box stays 95px from the top; at 375px it is a normal stacked block):



375px



665px



1280px

HTML

```
<main class="product">

    <figure class="product-image">
      <img src="images/iphone-17-pro-max.png" alt="iPhone 17 Pro Max">
      <figcaption>iPhone 17 Pro Max in Cosmic Orange</figcaption>
    </figure>

    <section class="product-details" aria-label="Product details">
      <h2>iPhone 17 Pro Max</h2>
      <p class="price">Rs. 470,000</p>
      <div class="qty">
        <label for="qty">Quantity</label>
        <input type="number" id="qty" name="qty" value="1" min="1" max="10">
      </div>
      <button type="button" class="add-btn">Add to Cart</button>
    </section>
```

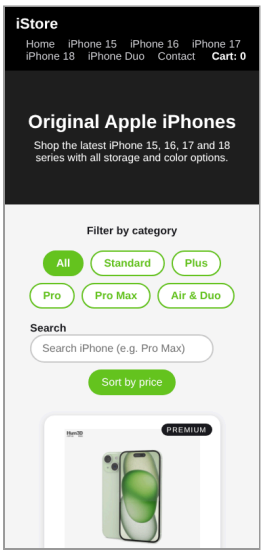
CSS

```
/* ===== TASK 2: CSS Grid with named areas ===== */
    /* Below BP: one column -> image, details, description */
    .product {
      display: grid;
      grid-template-columns: 1fr;
      grid-template-areas:
        "image"
        "details"
        "description";
      gap: 30px;
      max-width: 1100px;
      margin: 0 auto;
      padding: 30px 20px;
    }
    .product-image       { grid-area: image; }
    .product-details     { grid-area: details; }
    .product-description { grid-area: description; }

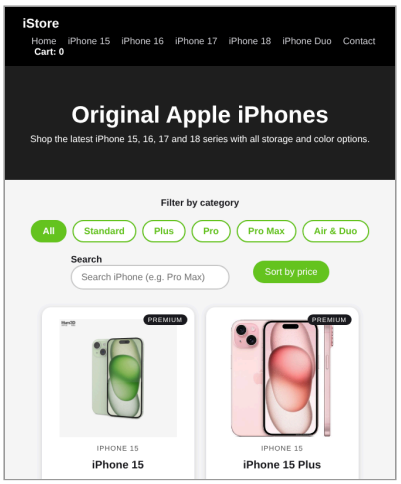
    /* From BP = 600 + 65 = 665px and wider: two columns, description full width underneath */
    @media (min-width: 665px) {
      .product {
        grid-template-columns: 1fr 1fr;
        grid-template-areas:
          "image details"
          "description description";
      }
      .product-details {
        align-self: start;        /* do not stretch, otherwise sticky has no room to move */
        position: sticky;
        top: 95px;                /* (N mod 40) + 70 = 25 + 70 = 95px */
      }
    }
```

Task 3 - JavaScript: live search and price sort

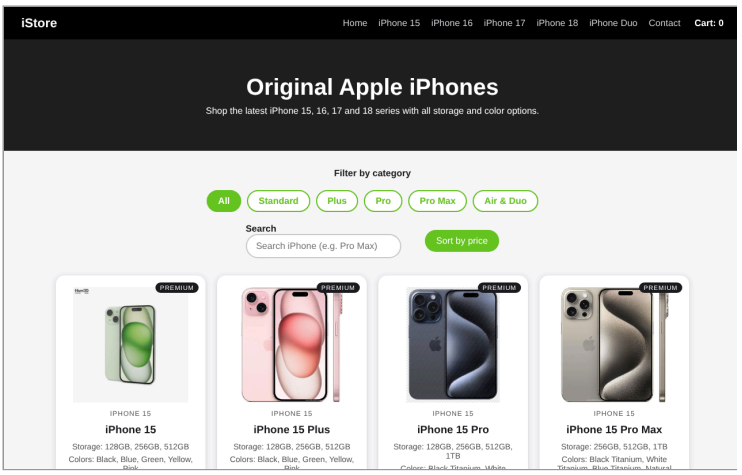
On page load (Premium labels visible):



375px

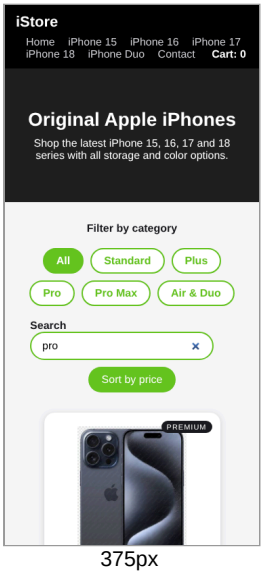


665px

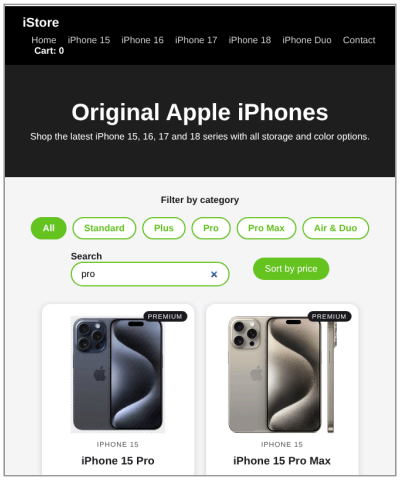


1280px

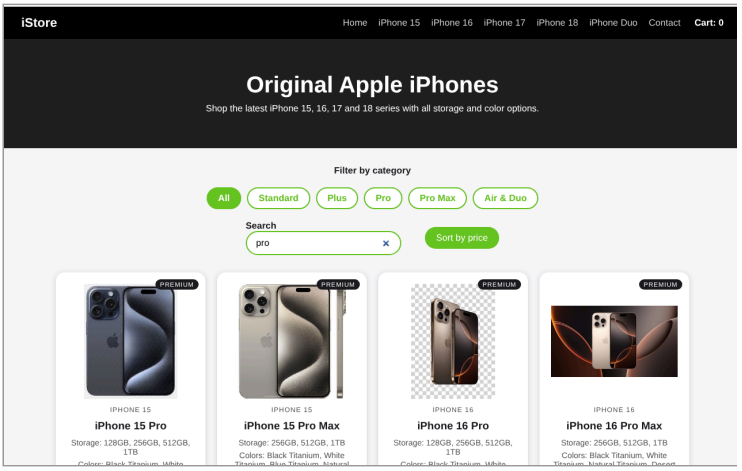
After typing "pro" in the search box:



375px

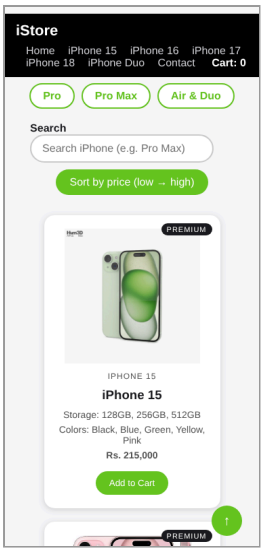


665px

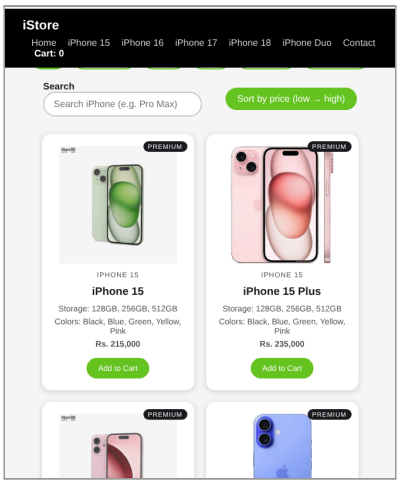


1280px

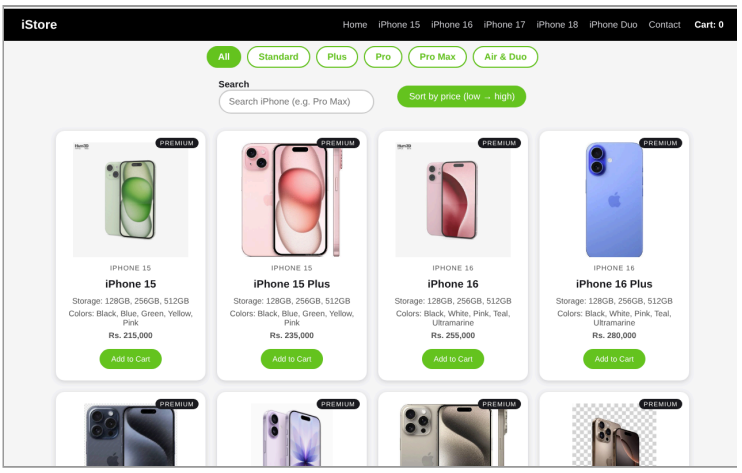
After clicking "Sort by price" (lowest to highest):



375px



665px



1280px

HTML added (search box, sort button, message)

```
<!-- TASK 3: search + sort -->
    <div class="toolbar">
      <form role="search" onsubmit="return false">
        <label for="search">Search</label>
        <input type="search" id="search" name="search" placeholder="Search iPhone (e.g. Pro Max)"
autocomplete="off">
      </form>
      <button type="button" id="sort-price">Sort by price</button>
    </div>

    <p id="no-results" hidden>No products found</p>
```

script.js

```
// Task 3: live search, price sort, Premium label (plain JavaScript)
// N = 65  ->  Premium limit = N x 100 = 6500

const PREMIUM_LIMIT = 6500;

const grid = document.getElementById("product-grid");
const searchInput = document.getElementById("search");
const sortButton = document.getElementById("sort-price");
const noResults = document.getElementById("no-results");
const categoryRadios = document.querySelectorAll('input[name="category"]');

// Read the price from the card itself: "Rs. 215,000" -> 215000
function getPrice(card) {
  return Number(card.querySelector(".price").textContent.replace(/[^0-9]/g, ""));
}

function getName(card) {
  return card.querySelector("h3").textContent.toLowerCase();
}

function getSelectedCategory() {
  return document.querySelector('input[name="category"]:checked').value;
}

// A card is visible only if it matches BOTH the search text and the category filter.
// The category is hidden by CSS (Task 1); here we only use the `hidden` attribute for
// the search, then count what is really visible to decide about the message.
function applyFilters() {
  const text = searchInput.value.trim().toLowerCase();
  const category = getSelectedCategory();
  let visibleCount = 0;

  grid.querySelectorAll(".card").forEach(function (card) {
    const matchesSearch = getName(card).includes(text);
    const matchesCategory = category === "all" || card.dataset.category === category;

    card.hidden = !matchesSearch;
    if (matchesSearch && matchesCategory) {
      visibleCount++;
    }
```

```
  });

  // "No products found" replaces the grid when nothing is visible
  noResults.hidden = visibleCount !== 0;
  grid.hidden = visibleCount === 0;
}

searchInput.addEventListener("input", applyFilters);
categoryRadios.forEach(function (radio) {
  radio.addEventListener("change", applyFilters);
});

// Sort by price: toggles low -> high, then high -> low, ...
let ascending = true;

sortButton.addEventListener("click", function () {
  const cards = Array.from(grid.querySelectorAll(".card"));

  cards.sort(function (a, b) {
    return ascending ? getPrice(a) - getPrice(b) : getPrice(b) - getPrice(a);
  });

  // appendChild MOVES an existing element, so this re-orders the real cards
  cards.forEach(function (card) {
    grid.appendChild(card);
  });

  sortButton.textContent = ascending ? "Sort by price (low → high)" : "Sort by price (high → low)";
  ascending = !ascending;
});

// "Premium" label for products above N x 100
grid.querySelectorAll(".card").forEach(function (card) {
  if (getPrice(card) > PREMIUM_LIMIT) {
    const label = document.createElement("span");
    label.className = "premium-label";
    label.textContent = "Premium";
    card.insertBefore(label, card.firstChild);
  }
});
```